CENTRO UNIVERSITÁRIO
CAMPO REAL
EXCELÊNCIA EM ENSINO SUPERIOR

Trabalho prático
Data Warehouse da Campo Real Energia

Extração de dados abertos · Qualidade de dados em Python · Star schema · Indicadores em Power BI ou Tableau

1. O cenário

A **Campo Real Energia** é uma geradora fictícia com nove usinas espalhadas pelos quatro submercados do Sistema Interligado Nacional: hidrelétricas, PCH, eólicas, solares, biomassa e uma térmica a gás. Toda a energia que ela produz pode ser valorada pelo **PLD** (Preço de Liquidação das Diferenças), o preço horário do mercado de curto prazo calculado pela CCEE para cada submercado.

A diretoria quer respostas para uma pergunta simples de fazer e difícil de responder:

> **Pergunta da diretoria**
> “Quanto a nossa geração vale, onde e quando ganhamos ou perdemos dinheiro, e podemos confiar nos números que nos mostram?”

Vocês são a equipe de dados. A tarefa é construir, do zero, o caminho entre os dados brutos e os indicadores que a diretoria vai usar: extrair, carregar, limpar, modelar e apresentar.

Formato: grupos de até 5 pessoas. Datas de entrega e de apresentação serão informadas pela professora.

2. O que vocês recebem

O grupo pode trabalhar com **MySQL** ou com **PostgreSQL**, à escolha. Há um script para cada banco: **campo_real_energia.sql** (MySQL 8) e **campo_real_energia_postgres.sql** (PostgreSQL 13 ou superior, pronto para executar no DBeaver com Alt+X). Os dois criam o mesmo banco de origem da empresa. Ele **não** é um data warehouse: são as tabelas operacionais de onde vocês vão extrair os dados.

Escolham um dos dois e usem o mesmo banco em todo o trabalho (origem, tabelas de PLD, métricas de qualidade e data warehouse).

Tabela	Conteúdo
submercado	Os quatro submercados do SIN (SE, S, NE, N).
fonte_energia	Tipos de fonte e se são renováveis ou intermitentes.
usina	Cadastro das usinas: fonte, submercado, UF, município, coordenadas, potência e início de operação.
feriado_nacional	Feriados nacionais de 2025.
parada_usina	Manutenções programadas e falhas, com início e fim.
medicao_geracao	Energia gerada por usina a cada hora de 2025, com disponibilidade e o momento em que o dado chegou (recebido_em).
log_carga	Registro das cargas executadas.

O preço (PLD) não está nesse banco. Criar a tabela de PLD e carregá-la com os dados do site da CCEE é responsabilidade do grupo (etapa 1).

> **Atenção**
> Como em qualquer empresa, esses dados **não são confiáveis de saída**. O sistema de medição falha, integrações reprocessam, pessoas digitam errado e cadastros são feitos às pressas. Não há chave única nem chave estrangeira na tabela de medição: o banco não protege nada. Encontrar e tratar esses problemas faz parte da nota.

3. O que vocês devem fazer

O trabalho tem quatro etapas. Elas não precisam ser feitas em sequência rígida, mas a entrega final precisa cobrir todas.

Etapa 1 · Extração obrigatória de fontes externas (Python)

Sem preço não há como saber quanto a geração vale. O grupo deve buscar os dados nas fontes públicas, com Python, e criar as tabelas para recebê-los:

- **Obrigatório A · PLD horário da CCEE.** O grupo deve **criar a(s) tabela(s) de PLD** no banco escolhido (camada raw e camada tratada, com tipos, chaves e grão definidos por vocês) e **inserir os dados extraídos do site**: PLD horário de 2025 e de 2026, até o último mês publicado, pela API do portal de dados abertos. Download manual de CSV ou importação pelos assistentes do Workbench ou do DBeaver **não** valem.
- **Obrigatório B · pelo menos uma fonte complementar**, escolhida pelo grupo:
  – **Open-Meteo**: clima horário (radiação, vento, temperatura, chuva) nas coordenadas das usinas;
  – **ONS**: carga, geração por fonte, ENA ou reservatórios por submercado;
  – **INMET**: dados das estações meteorológicas automáticas.

Requisitos da extração:

- Guardar o dado **exatamente como chegou** (camada raw), com data e hora da extração e a origem.
- Tratar falhas: tempo limite, nova tentativa, resposta vazia, mudança de colunas e limite de requisições.

- Registrar cada execução em uma tabela de log (início, fim, linhas lidas, linhas gravadas, status, erro).
- Nada de credenciais ou caminhos fixos de máquina no código: usar arquivo de configuração.

Ponto de partida para a API da CCEE (padrão CKAN):

```
import requests
URL = "https://dadosabertos.ccee.org.br/api/3/action/datastore_search"
params = {"resource_id": "ID_DO_RECURSO_PLD_HORARIO_2025", "limit": 5000, "offset": 0}
r = requests.get(URL, params=params, timeout=60)
dados = r.json()["result"]["records"]   # repetir aumentando o offset até acabar
```

O *resource_id* de cada ano aparece na página de cada recurso do conjunto PLD_HORARIO. Descobrir como paginar e quando parar é parte da tarefa.

Etapa 2 · Qualidade de dados (Python)

Usando o que vimos nas aulas anteriores (pandas, profiling e regras de validação), o grupo deve avaliar **todas** as tabelas de origem e as fontes externas antes de carregar o warehouse.

Dimensão	Pergunta que a regra responde
Completude	Falta algum valor, alguma hora, algum dia, algum campo de cadastro?
Unicidade	O mesmo fato aparece mais de uma vez? A mesma usina está cadastrada duas vezes?
Validade	O valor está dentro do domínio e das regras (sinal, faixa, datas, limites regulatórios)?
Consistência	Os dados concordam entre si e com o mundo físico (entre colunas, entre tabelas)?
Acurácia	O valor é plausível? Há outliers, unidades erradas, sensores com comportamento estranho?
Integridade referencial	Todo registro aponta para algo que existe no cadastro?
Atualidade	O dado chegou a tempo? Alguma usina parou de enviar dados?

Requisitos:

- Profiling de cada tabela (tipos, nulos, distintos, mínimos, máximos, distribuições).
- Pelo menos **uma regra por dimensão**, escritas como funções reutilizáveis, não como células soltas.
- Resultados gravados em uma **tabela de métricas de qualidade** no banco (regra, dimensão, tabela, registros avaliados, registros com falha, percentual, data da execução). Prints não contam como evidência.
- Para cada problema encontrado, uma decisão justificada: **corrigir**, **descartar**, **mandar para quarentena** ou **apenas sinalizar**. Corrigir sem justificar é inventar dado.
- Relatório antes x depois: quanto cada indicador de negócio muda depois da limpeza.

> **Dica**
> Alguns problemas só aparecem quando vocês cruzam tabelas, olham sequências no tempo ou comparam com a física da usina. Regras que só checam se o campo está vazio vão deixar muita coisa passar.

Etapa 3 · Modelagem dimensional (star schema no MySQL ou PostgreSQL)

Com os dados tratados, o grupo projeta e constrói o data warehouse em um schema separado (por exemplo, **dw_campo_real**). O modelo é de vocês; os requisitos mínimos são:

- **Matriz de barramento** relacionando processos de negócio e dimensões.
- **Grão declarado** por escrito para cada tabela fato, antes de criar a tabela.
- Pelo menos **duas tabelas fato**: uma de geração e uma de preço. A fonte externa escolhida deve entrar no modelo (como fato próprio ou como atributo, conforme o grão).
- **Dimensão de tempo construída pelo grupo**, com os atributos necessários para responder às perguntas (hora, dia útil, feriado, mês, estação do ano, patamar de carga, o que mais acharem útil).
- **Chaves substitutas** nas dimensões e uma estratégia para registros sem correspondência (membro “desconhecido”).
- Decisão sobre **mudanças no cadastro** (SCD): se uma usina mudar de nome, de potência ou de situação, o histórico deve ser preservado? Justifiquem o tipo escolhido.
- Tratamento explícito das **granularidades diferentes** (dado horário, diário e cadastral) e de como associar uma estação ou ponto de clima a uma usina ou submercado.
- Script SQL de criação (DDL) e de carga versionados no repositório.

Etapa 4 · Indicadores em Power BI ou Tableau

O painel deve ler **somente do data warehouse**, nunca das tabelas de origem. Ele precisa responder às perguntas abaixo. As cinco primeiras são obrigatórias; escolham pelo menos uma das duas últimas.

#	Pergunta de negócio	Indicadores sugeridos
1	Quanto geramos por usina, fonte e submercado ao longo do ano?	Geração (MWh), fator de capacidade (%), evolução mensal.
2	Quanto a nossa geração vale a preço de mercado?	Receita valorada ao PLD = Σ (geração da hora × PLD da hora no submercado da usina).
3	Qual fonte captura o melhor preço?	Preço capturado = receita valorada ÷ geração; razão de captura = preço capturado ÷ PLD médio do submercado.
4	Em que horas e meses o PLD é mais alto, e nós geramos nesses momentos?	Mapa de calor hora × mês do PLD; perfil horário de geração por fonte.
5	Quanto perdemos com paradas?	Horas indisponíveis, energia e receita estimadas não geradas, por tipo de parada.
6	O clima ou o sistema explicam a nossa geração e o preço?	Correlação radiação × solar, vento × eólica, ou carga/ENA do ONS × PLD (conforme a fonte escolhida).
7	Podemos confiar nesses números?	Índice de qualidade por dimensão e por tabela; diferença da receita antes e depois da limpeza.

O painel será apresentado à “diretoria” em formato de história: comecem pela pergunta, mostrem a evidência e terminem com uma recomendação.

4. Entregáveis

- **Repositório Git** com README que permita a qualquer pessoa reproduzir tudo: requisitos, configuração, ordem de execução.
- Código Python de extração, carga e qualidade; scripts SQL do warehouse.
- **Documento de modelagem**: matriz de barramento, grão de cada fato, diagrama do star schema, decisões de SCD.
- **Relatório de qualidade**: problemas encontrados por dimensão, quantidade afetada, decisão tomada e impacto.
- Arquivo do painel (.pbix ou .twbx) e apresentação de 10 a 15 minutos.

5. Avaliação

Critério	Peso	O que será observado
Extração e carga	25%	Automação real pela API, raw preservado, tabelas de PLD bem definidas, tratamento de falhas e logs.
Qualidade de dados	20%	Cobertura das dimensões, regras reutilizáveis, métricas em tabela, decisões justificadas, impacto medido.
Modelagem dimensional	25%	Grão bem definido, dimensões adequadas, chaves substitutas, SCD, tratamento de granularidades.
Indicadores e painel	20%	Perguntas respondidas corretamente, leitura do DW, clareza visual.
Documentação e apresentação	10%	Reprodutibilidade, clareza do README, narrativa para a diretoria.

6. Fontes de dados

Fonte	Endereço	Observação
CCEE · PLD horário	dadosabertos.ccee.org.br/dataset/pld_horario	Um recurso por ano; API CKAN em dadosabertos.ccee.org.br/api/3. Licença CC-BY 4.0.
Open-Meteo · histórico	archive-api.open-meteo.com/v1/archive Documentação: open-meteo.com/en/docs/historical-weather-api	Parâmetros: latitude, longitude, start_date, end_date, hourly, timezone. Uso não comercial com atribuição.
ONS · dados abertos	dados.ons.org.br Bucket S3: s3://ons-aws-prod-opendata (sa-east-1)	Carga, geração, ENA e reservatórios. Licença CC-BY 4.0.
INMET · estações	portal.inmet.gov.br/dadoshistoricos portal.inmet.gov.br/paginas/catalogoaut	Arquivos anuais com dados horários das estações automáticas; confiram o fuso horário.

Regras do jogo

- Não alterem as tabelas de origem para “consertar” dados: a limpeza acontece no caminho até o warehouse.
- Toda fonte externa deve ser citada no painel e no README, conforme a licença.
- Respeitem os limites de requisição das APIs: guardem o que já foi baixado em vez de baixar tudo de novo.
- Se uma fonte estiver fora do ar, documentem o que aconteceu e como o processo se comportou. Isso também é avaliado.